

Lecture 18 - Custom NPU Dialect

Topic: npu.matmul / npu.dma / npu.barrier ODS design

Goal: turn hardware contracts into verifiable MLIR ops

Output: ODS skeleton, verifier checklist, FileCheck tests

오늘의 위치

1-17 강 : IR, dialect, pass, bufferization, vector, transform, quant, fusion

18 강 : target-specific NPU dialect bring-up

19 강 : runtime/ABI/codegen

Learning Objectives

Explain why custom NPU dialect is needed

Define op contracts with ODS/TableGen

Design npu.matmul, npu.dma, npu.barrier

Plan verifier and tests

Why Custom Dialect?

Generic IR cannot encode every target hazard

NPU kernels need tile shape, memory space, DMA order

Dialect provides namespace + target contract

Bad dialect design causes brittle lowering

Boundary Principle

npu_graph: semantic fusion/layout/quant

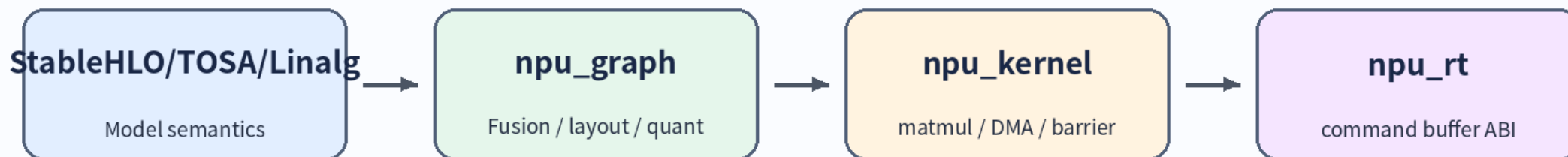
npu_kernel: tile/DMA/barrier/accumulator

npu_rt: command buffer and descriptor ABI

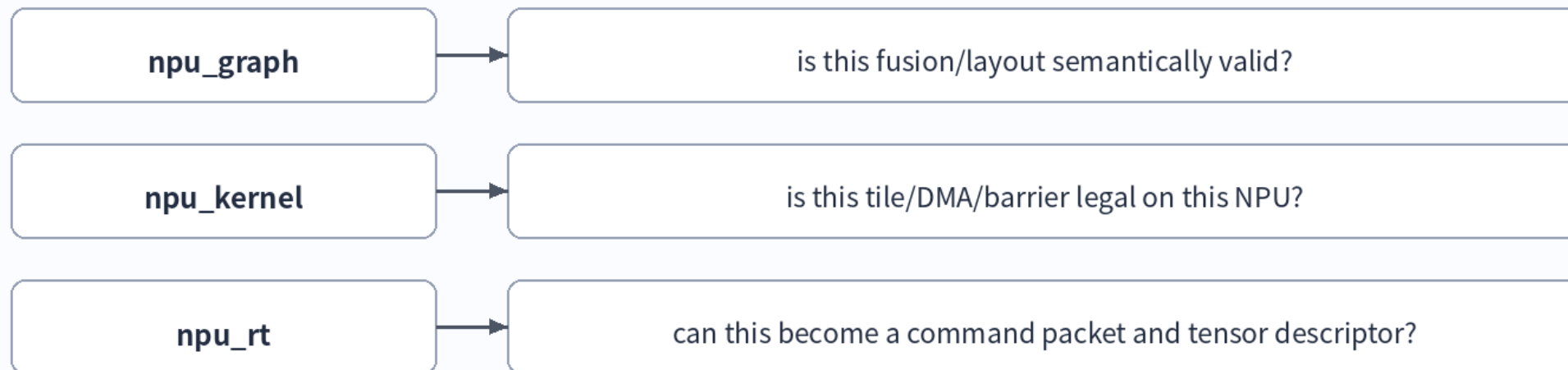
Do not mix packet binary details too early

NPU dialect layer diagram

Custom NPU Dialect Layering



What each layer verifies



ODS = Single Source of Truth

Operation name and namespace

Operands/results/attributes

Traits/interfaces

Assembly format

Verifier hook

ODS Anatomy

ODS / TableGen Anatomy

```
def NPU_MatmulOp : NPU_Op<"matmul", [MemoryEffectsOpInterface]> {  
  let arguments = (ins NPU_TileBuffer:$lhs, NPU_TileBuffer:$rhs,  
    NPU_AccBuffer:$acc, NPU_TileShapeAttr:$tile,  
    NPU_LayoutAttr:$lhs_layout, NPU_LayoutAttr:$rhs_layout);  
  let results = (outs NPU_AccBuffer:$result);  
  let assemblyFormat = "$lhs ` ` $rhs ` ` $acc attr-dict `:` ` ...";  
  let hasVerifier = 1;  
}
```

op name + namespace

operands/results typed by contract

attributes hold static target facts

assemblyFormat = readable IR

verifier = hardware legality

Minimal File Layout

include/NPU/IR/NPUOps.td
include/NPU/IR/NPUTypes.td
lib/NPU/IR/NPUDialect.cpp
lib/NPU/IR/NPUOps.cpp
test/NPU/IR/*.mlir

npu.matmul Contract

```
def NPU_MatmulOp : NPU_Op<"matmul", [MemoryEffectsOpInterface]> {  
  let arguments = (ins NPU_TileBuffer:$lhs,  
                    NPU_TileBuffer:$rhs,  
                    NPU_AccBuffer:$acc,  
                    NPU_TileShapeAttr:$tile,  
                    NPU_LayoutAttr:$lhs_layout,  
                    NPU_LayoutAttr:$rhs_layout);  
  let results = (outs NPU_AccBuffer:$result);  
  let hasVerifier = 1;  
}
```

npu.matmul Must Verify

M/N/K supported by target engine

element type pair is legal

lhs/rhs layout matches dataflow

accumulator width and epilogue policy explicit

npu.dma Contract

```
def NPU_DmaOp : NPU_Op<"dma", [MemoryEffectsOpInterface]> {  
  let arguments = (ins AnyType:$source, AnyType:$dest,  
                    NPU_DmaDirectionAttr:$direction,  
                    I64Attr:$elements,  
                    OptionalAttr<I64Attr>:$stride);  
  let results = (outs NPU-Token:$token);  
  let hasVerifier = 1;  
}
```

npu.dma Design Choices

Token model: easy dependency analysis

Tag-buffer model: close to `memref.dma_start/wait`

Token can lower to hardware tag later

Be explicit: bytes vs elements

npu.barrier Contract

```
def NPU_BarrierOp : NPU_Op<"barrier", [MemoryEffectsOpInterface]> {  
  let arguments = (ins Variadic<NPU-Token>:$tokens,  
                    NPU_BarrierScopeAttr:$scope);  
  let results = (outs);  
  let assemblyFormat = "$tokens attr-dict `:` ` type($tokens)";  
  let hasVerifier = 1;  
}
```

Operation Contracts Summary

npu.matmul / npu.dma / npu.barrier Contracts

npu.matmul

- tile shape
- lhs/rhs layout
- accumulator type
- epilogue policy
- read/write effects

Verifier rejects illegal hardware contracts

npu.dma

- src/dst memory space
- elements or bytes
- stride policy
- async token
- alignment limits

Verifier rejects illegal hardware contracts

npu.barrier

- token operands
- scope attribute
- ordering semantics
- deadlock checks
- region boundary

Verifier rejects illegal hardware contracts

Types and Attributes

`!npu.tile_buffer, !npu.acc_buffer, !npu.token`

`#npu.tile<m,n,k>`

`#npu.layout<mk|kn|blocked>`

`#npu.mem_space<dram|sram|accum>`

Attribute vs Operand

Attribute: static hardware fact

Operand: dynamic/data-dependent value

Type: storage/value contract

Wrong classification hurts canonicalization

Traits and Interfaces

MemoryEffectsOpInterface for command effects
InferTypeOpInterface when result type is derivable
Custom interfaces for target capability queries
Traits for structural promises

Assembly Format

Readable IR improves debugging

Generic form helps diagnose parser bugs

Expose important contract fields

Avoid opaque attr-dict-only design

Verifier Strategy

Reject illegal IR, not slow IR

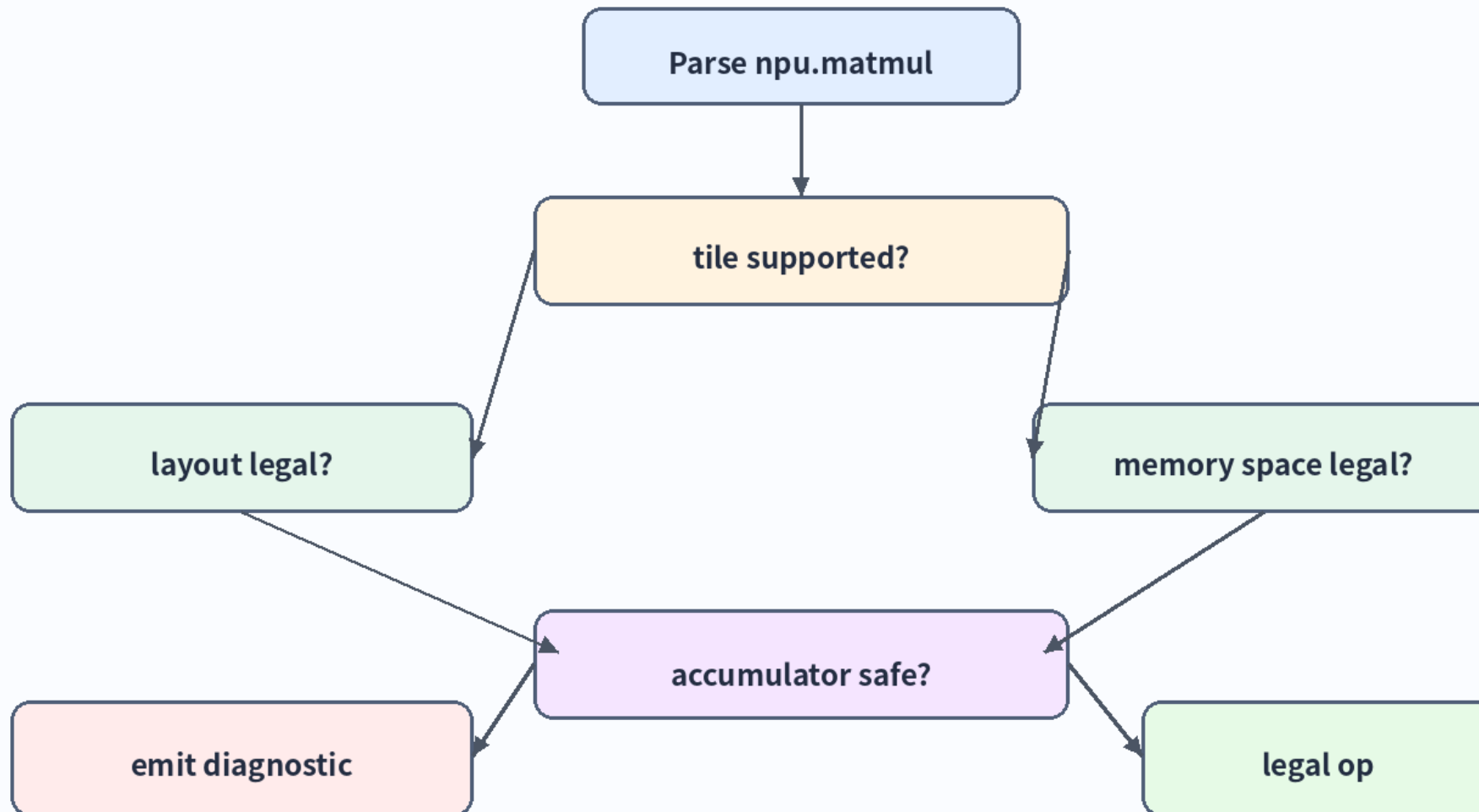
Check shape/type/layout first

Check target capability last

Emit diagnostics with failed HW rule

Verifier Decision Tree

Verifier Decision Tree



Canonicalization Hooks

Remove redundant barriers only with dependency proof

Fold repeated static attrs

Canonicalize layout aliases

Do not reorder commands blindly

Testing Matrix

Parser/printer round-trip

Generic form test

Negative verifier diagnostics

Lowering pass tests

Command hazard tests

Lowering into NPU Dialect

vector.contract -> npu.matmul

memref/subview/copy -> npu.dma when crossing memory spaces

dependency analysis -> npu.barrier

npu commands -> runtime packet in lecture 19

Case Study: MatMul Tile

M=16, N=16, K=32

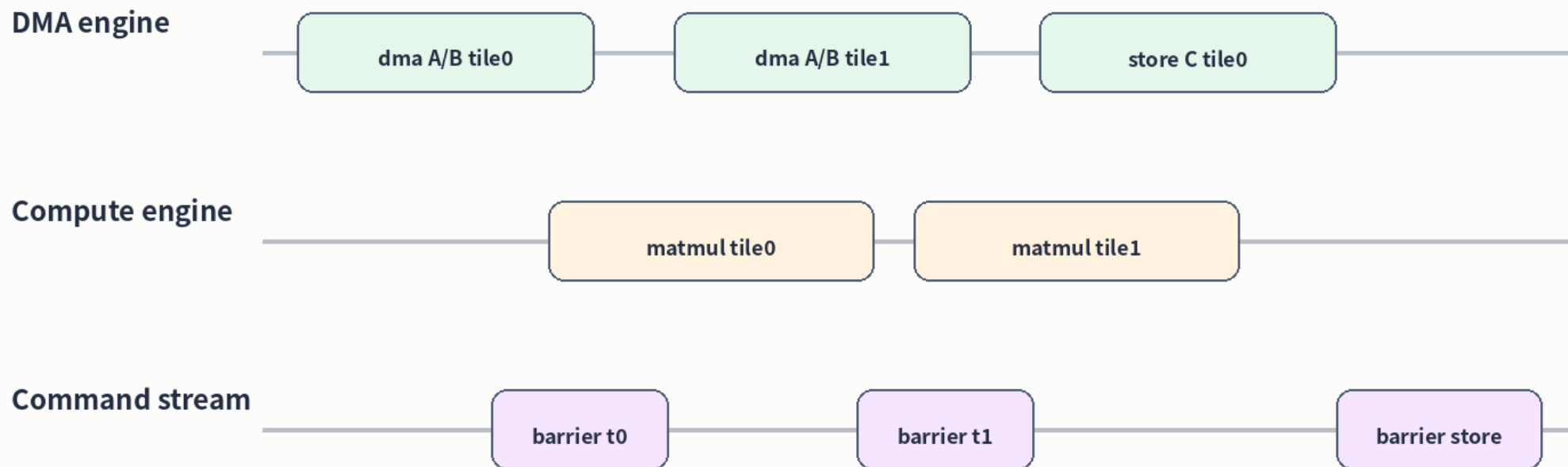
lhs: SRAM tile layout mk

rhs: SRAM tile layout kn

acc: ACCUM tile i32

DMA + Barrier Timeline

DMA + Barrier + MatMul Timeline



Design rule: expose tokens/barriers early enough to verify hazards, but late enough to preserve graph-level optimization.

CMake and Build

`add_mlir_dialect(NPUOps npu)`

`mlir_tablegen` for op declarations/definitions

Dialect library depends on IncGen target

Tool registers the dialect

Common Mistake #1

Putting profitability in verifier
Verifier should reject illegal IR only
Cost model should choose schedules
Fallback should remain possible

Common Mistake #2

Encoding packet binary too early

Use semantic command fields first

Lower packet format at runtime boundary

Keeps transformations readable

Common Mistake #3

Implicit DMA/compute ordering

Use token/barrier contract explicitly

Avoid depending on operation order only

Test hazards in CI

Exercise A

Classify fields for npu.matmul

Mark verifier fields

Mark cost model fields

Exercise B

Write ODS skeleton for npu.dma

Choose token vs tag-buffer model

Add memory effect interface plan

Exercise C

Write positive parser/printer test

Write negative unsupported tile test

Write negative empty barrier test

Takeaways

Custom dialect is a target contract

ODS is the source of truth

Verifier turns HW limits into compiler diagnostics

npu.matmul/dma/barrier prepare runtime codegen

References

MLIR Defining Dialects

MLIR ODS

MLIR Creating a Dialect

MLIR MemRef Dialect

MLIR Testing Guide